영지식증명 톺아보기 3편: 프로그램을 다항식으로 - 산술화

Author: 김대익
Reviewed By: —

Disclaimer: 본 글은 영지식증명에서 가장 핵심적인 과정인 "산술화(Arithmetization)"를 다룹니다. 프로그램이 어떻게 다항식으로 변환되는지, 그 과정에서 왜 R1CS, 라그랑주 보간, FFT가 필요한지 설명합니다.

---

1. 들어가며: 산술화란?

1편에서 "증명하고 싶은 계산을 산술 회로로 표현한다"고 했습니다. 2편에서 유한체와 타원곡선이라는 수학적 도구를 다뤘습니다.

이제 핵심 질문을 다룹니다:

> **"어떻게 프로그램을 다항식으로 바꾸는가?"**

이 과정을 **산술화(Arithmetization)**라고 합니다. 산술화는 ZKP 전체 파이프라인에서 가장 중요한 단계입니다[^1]:

```
코드 → 평탄화(Flattening) → R1CS → QAP(다항식) → 증명 생성
```

왜 다항식인가? 1편에서 간단히 언급했지만, 다시 정리합니다:

1. **압축**: n개의 제약 조건을 하나의 다항식으로 표현
2. **검증 효율성**: Schwartz-Zippel 보조정리로 O(1) 시간에 등가성 검사
3. **암호학적 숨김**: 다항식 커밋먼트로 값을 숨기면서 증명 가능

이번 편에서 다루는 것:

- **평탄화**: 복잡한 코드를 단순한 게이트로 분해
- **R1CS**: 게이트를 행렬 형태의 제약 조건으로 표현
- **QAP**: R1CS를 다항식으로 변환 (라그랑주 보간)
- **FFT**: 다항식 연산을 O(n log n)으로 최적화

[^1]: Buterin, V. (2016). *Quadratic Arithmetic Programs: from Zero to Hero*. Medium.

---

2. 평탄화: 코드를 게이트로 분해

2.1 예제: $x^3 + x + 5 = 35$

Vitalik의 유명한 예제를 사용합니다[^1]. 우리가 증명하고 싶은 것:

> "나는 $x^3 + x + 5 = 35$를 만족하는 x를 알고 있다" (정답: x = 3)

이 식을 직접 증명할 수는 없습니다. 먼저 **단순한 연산으로 분해**해야 합니다.

규칙: 각 문장은 최대 하나의 곱셈만 포함해야 합니다.
(덧셈은 여러 개 가능)

```
원래: x³ + x + 5 = 35

분해 후:
sym_1 = x * x       // x²
y = sym_1 * x       // x³
sym_2 = y + x       // x³ + x
out = sym_2 + 5     // x³ + x + 5
```

이렇게 분해하면 4개의 "게이트"가 생깁니다. 각 게이트는 `left ∘ right = output` 형태입니다.

2.2 왜 곱셈만 분리하는가?

덧셈은 선형 연산입니다. 여러 덧셈을 하나의 선형 조합으로 묶을 수 있습니다:

```
a + b + c + d = (1·a + 1·b + 1·c + 1·d)
```

곱셈은 비선형 연산입니다. 곱셈은 "차수를 높이는" 연산이므로 별도로 처리해야 합니다.

R1CS는 "Rank-**1**"입니다. 이는 각 제약 조건에서 곱셈이 **딱 한 번**만 등장한다는 의미입니다.

2.3 위트니스(Witness)

평탄화된 회로를 실행하면 모든 중간 값을 알 수 있습니다. 이 값들의 집합을 **위트니스(Witness)** 또는 **솔루션 벡터**라고 합니다.

x = 3일 때:

```
1 = 1       (상수)
x = 3
out = 35
sym_1 = 9   (3 × 3)
y = 27      (9 × 3)
sym_2 = 30  (27 + 3)
```

위트니스 벡터: **s = [1, x, out, sym_1, y, sym_2] = [1, 3, 35, 9, 27, 30]**

(첫 번째 원소 1은 상수 항을 표현하기 위해 필요합니다)

---

3. R1CS: 제약 조건을 행렬로

3.1 R1CS의 정의

R1CS (Rank-1 Constraint System)는 세 벡터의 삼중쌍 (a, b, c)와 솔루션 벡터 s로 이루어지며, 다음을 만족합니다[^2]:

```
(a · s) × (b · s) = (c · s)
```

여기서 `·`는 **내적(dot product)**입니다.

직관적으로:

- **a**: 곱셈의 왼쪽 입력을 선택
- **b**: 곱셈의 오른쪽 입력을 선택
- **c**: 출력을 선택

[^2]: 0xPARC. *R1CS Explainer*. https://learn.0xparc.org/materials/circom/additional-learning-resources/r1cs%20explainer/

3.2 첫 번째 게이트: x × x = sym_1

게이트: `sym_1 = x * x`

위트니스 벡터의 순서: s = [1, x, out, sym_1, y, sym_2]
인덱스: [0, 1, 2, 3, 4, 5]

왼쪽(a): x를 선택 → a = [0, 1, 0, 0, 0, 0]

```
a · s = 0×1 + 1×3 + 0×35 + 0×9 + 0×27 + 0×30 = 3
```

오른쪽(b): x를 선택 → b = [0, 1, 0, 0, 0, 0]

```
b · s = 3
```

출력(c): sym_1을 선택 → c = [0, 0, 0, 1, 0, 0]

```
c · s = 0×1 + 0×3 + 0×35 + 1×9 + 0×27 + 0×30 = 9
```

검증: (a · s) × (b · s) = 3 × 3 = 9 = (c · s) ✓

3.3 두 번째 게이트: sym_1 × x = y

게이트: `y = sym_1 * x`

a: sym_1 선택 → a = [0, 0, 0, 1, 0, 0]
b: x 선택 → b = [0, 1, 0, 0, 0, 0]
c: y 선택 → c = [0, 0, 0, 0, 1, 0]

검증: 9 × 3 = 27 ✓

3.4 세 번째 게이트: (y + x) × 1 = sym_2

게이트: `sym_2 = y + x`

덧셈은 곱셈이 아닙니다. 하지만 **"(y + x) × 1 = sym_2"**로 표현하면 R1CS 형태가 됩니다!

a: y + x 선택 → a = [0, 1, 0, 0, 1, 0]
b: 상수 1 선택 → b = [1, 0, 0, 0, 0, 0]
c: sym_2 선택 → c = [0, 0, 0, 0, 0, 1]

검증:

- a · s = 0 + 3 + 0 + 0 + 27 + 0 = 30
- b · s = 1
- c · s = 30
- 30 × 1 = 30 ✓

3.5 네 번째 게이트: (sym_2 + 5) × 1 = out

게이트: `out = sym_2 + 5`

a: sym_2 + 5 선택 → a = [5, 0, 0, 0, 0, 1] (상수 5는 첫 번째 원소에) **b**: 상수 1 선택 → b = [1, 0, 0, 0, 0, 0]
c: out 선택 → c = [0, 0, 1, 0, 0, 0]

검증:

- a · s = 5×1 + 0 + 0 + 0 + 0 + 30 = 35
- b · s = 1
- c · s = 35
- 35 × 1 = 35 ✓

3.6 행렬 형태의 R1CS

4개의 제약 조건을 행렬로 정리합니다:

```
      [1, x, out, sym_1, y, sym_2]

A = | 0  1  0    0     0    0 |   (게이트 1: x)
    | 0  0  0    1     0    0 |   (게이트 2: sym_1)
    | 0  1  0    0     1    0 |   (게이트 3: y + x)
    | 5  0  0    0     0    1 |   (게이트 4: sym_2 + 5)

B = | 0  1  0    0     0    0 |   (게이트 1: x)
    | 0  1  0    0     0    0 |   (게이트 2: x)
    | 1  0  0    0     0    0 |   (게이트 3: 1)
    | 1  0  0    0     0    0 |   (게이트 4: 1)

C = | 0  0  0    1     0    0 |   (게이트 1: sym_1)
    | 0  0  0    0     1    0 |   (게이트 2: y)
```

```
      | 0  0  0    0     0    1 |   (게이트 3: sym_2)
      | 0  0  1    0     0    0 |   (게이트 4: out)
```

R1CS의 본질: 솔루션 벡터 s가 유효하면:

```
(As) ⊙ (Bs) = Cs
```

여기서 ⊙는 **원소별 곱셈(Hadamard product)**입니다.

---

4. 왜 행렬인가?: n개의 제약을 하나로

4.1 개별 검증의 비효율성

4개의 제약 조건을 개별적으로 검증하면:

- 제약 1: $(a_1 \cdot s) \times (b_1 \cdot s) = (c_1 \cdot s)$?
- 제약 2: $(a_2 \cdot s) \times (b_2 \cdot s) = (c_2 \cdot s)$?
- 제약 3: $(a_3 \cdot s) \times (b_3 \cdot s) = (c_3 \cdot s)$?
- 제약 4: $(a_4 \cdot s) \times (b_4 \cdot s) = (c_4 \cdot s)$?

실제 ZKP 회로는 **수백만 개의 제약 조건**을 가집니다. 각각을 검증하면 O(n) 시간이 걸립니다.

4.2 행렬의 역할

행렬 형태로 묶으면:

```
(As) ⊙ (Bs) = Cs
```

하나의 방정식으로 모든 제약을 표현합니다. 하지만 여전히 벡터 연산이므로 검증은 O(n)입니다.

진정한 효율성은 **다항식 변환**에서 옵니다.

---

5. QAP: 행렬을 다항식으로

5.1 핵심 아이디어

QAP (Quadratic Arithmetic Program)은 R1CS의 행렬을 다항식으로 변환합니다[^3].

핵심 통찰:

> "행렬의 각 열을 다항식으로 변환하면, n개의 제약 조건을 **하나의 다항식 방정식**으로 표현할 수 있다."

[^3]: Gennaro, R., et al. (2013). *Quadratic Span Programs and Succinct NIZKs without PCPs*. EUROCRYPT 2013.

5.2 라그랑주 보간법 (Lagrange Interpolation)

n개의 점 $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$이 주어지면, 이 점들을 모두 지나는 **유일한** 다항식 $L(x)$를 찾을 수 있습니다 [^4]. 이 다항식의 차수는 최대 n-1입니다.

라그랑주 기저 다항식:

```
ℓᵢ(x) = ∏(j≠i) (x - xⱼ) / (xᵢ - xⱼ)
```

이 기저 다항식은 특별한 성질을 갖습니다:

- $\ell_i(x_i) = 1$
- $\ell_i(x_j) = 0$ $(i \neq j)$

보간 다항식:

```
L(x) = Σᵢ yᵢ · ℓᵢ(x)
```

[^4]: Wikipedia. *Lagrange polynomial*. https://en.wikipedia.org/wiki/Lagrange_polynomial

5.3 예제: 행렬의 첫 번째 열을 다항식으로

행렬 A의 첫 번째 열: [0, 0, 0, 5]

이 열을 다항식으로 변환합니다. 평가 점을 x = 1, 2, 3, 4로 선택합니다.

점들:

- (1, 0)
- (2, 0)
- (3, 0)
- (4, 5)

라그랑주 보간으로 이 점들을 지나는 다항식 $A_0(x)$를 구합니다:

```
from sympy import *
x = symbols('x')

points = [(1,0), (2,0), (3,0), (4,5)]

# 라그랑주 보간
def lagrange_basis(points, i, x):
    xi, yi = points[i]
    result = 1
    for j, (xj, yj) in enumerate(points):
        if i != j:
            result *= (x - xj) / (xi - xj)
    return result

def lagrange_interp(points, x):
```

```
    return sum(yi * lagrange_basis(points, i, x)
               for i, (xi, yi) in enumerate(points))

A0 = expand(lagrange_interp(points, x))
print(A0)  # 5*x**3/6 - 5*x**2 + 55*x/6 - 5
```

결과: $A_0(x) = (5/6)x^3 - 5x^2 + (55/6)x - 5$

검증:

- $A_0(1) = 5/6 - 5 + 55/6 - 5 = 60/6 - 10 = 0$ ✓
- $A_0(2) = 5\times8/6 - 20 + 110/6 - 5 = 150/6 - 25 = 0$ ✓
- $A_0(3) = 5\times27/6 - 45 + 165/6 - 5 = 300/6 - 50 = 0$ ✓
- $A_0(4) = 5\times64/6 - 80 + 220/6 - 5 = 540/6 - 85 = 5$ ✓

5.4 모든 열을 다항식으로

행렬 A의 6개 열 각각에 대해 다항식을 구합니다:

- $A_0(x)$: 상수 항 계수
- $A_1(x)$: x 계수
- $A_2(x)$: out 계수
- $A_3(x)$: sym_1 계수
- $A_4(x)$: y 계수
- $A_5(x)$: sym_2 계수

마찬가지로 B와 C에 대해서도 6개씩의 다항식을 구합니다.

총 **18개의 다항식**이 생성됩니다 (각 차수 ≤ 3).

5.5 QAP 방정식

솔루션 벡터 $s = [s_0, s_1, s_2, s_3, s_4, s_5]$를 사용하여:

```
A(x) = s₀·A₀(x) + s₁·A₁(x) + s₂·A₂(x) + s₃·A₃(x) + s₄·A₄(x) + s₅·A₅(x)
B(x) = s₀·B₀(x) + s₁·B₁(x) + s₂·B₂(x) + s₃·B₃(x) + s₄·B₄(x) + s₅·B₅(x)
C(x) = s₀·C₀(x) + s₁·C₁(x) + s₂·C₂(x) + s₃·C₃(x) + s₄·C₄(x) + s₅·C₅(x)
```

핵심 성질: x = 1, 2, 3, 4에서 평가하면 원래 R1CS 제약을 얻습니다!

예: x = 1에서

```
A(1) = s₀·A₀(1) + s₁·A₁(1) + ... = s · a₁ = (a₁ · s)
B(1) = s · b₁ = (b₁ · s)
C(1) = s · c₁ = (c₁ · s)
```

따라서 A(1) × B(1) = C(1)은 첫 번째 R1CS 제약과 동치입니다.

5.6 소거 다항식 (Vanishing Polynomial)

소거 다항식 Z(x)는 모든 평가 점에서 0이 되는 다항식입니다:

```
Z(x) = (x - 1)(x - 2)(x - 3)(x - 4)
```

핵심 정리:

> 솔루션 s가 모든 R1CS 제약을 만족한다
> ⟺ A(x) × B(x) - C(x)가 x = 1, 2, 3, 4에서 모두 0이다
> ⟺ A(x) × B(x) - C(x)가 Z(x)로 나누어 떨어진다
> ⟺ **H(x) = (A(x) × B(x) - C(x)) / Z(x)가 다항식이다**

증명자(Prover)는 H(x)가 다항식임을 보여주면 됩니다!

---

6. Schwartz-Zippel: O(1) 검증의 비밀

6.1 다항식 동등성 검사의 어려움

두 다항식 P(x)와 Q(x)가 같은지 확인하려면:

- **순진한 방법**: 모든 계수 비교 → O(n) 시간
- **더 좋은 방법**: 있을까?

6.2 Schwartz-Zippel 보조정리

정리 (Schwartz-Zippel)[^5]: 차수 d인 0이 아닌 다항식 P(x)에 대해, 유한체 F의 원소를 무작위로 선택하면:

```
Pr[P(r) = 0] ≤ d / |F|
```

직관: 차수 d인 다항식은 최대 d개의 근을 가집니다. 따라서 |F|개의 원소 중 d개만이 근이 될 수 있으므로, 무작위로 근을 "맞출" 확률은 매우 낮습니다.

[^5]: Wikipedia. *Schwartz–Zippel lemma*. https://en.wikipedia.org/wiki/Schwartz%E2%80%93Zippel_lemma

6.3 ZKP에서의 활용

두 다항식 P(x)와 Q(x)가 같은지 확인하려면:

1. 검증자가 무작위 점 r을 선택
2. P(r)과 Q(r)을 계산
3. P(r) = Q(r)이면 "같음"으로 판정

오류 확률: d / |F|

예: BLS12-381에서 |F| ≈ 2^381, 회로 크기 n = 2^20이면:

```
Pr[오류] ≤ 2^20 / 2^381 ≈ 2^-361 ≈ 0
```

실질적으로 **무시할 수 있는** 확률입니다.

6.4 n개의 제약 조건, O(1) 검증

QAP 검증은 다음을 확인합니다:

```
A(r) × B(r) - C(r) = H(r) × Z(r)
```

- **왼쪽**: A(r), B(r), C(r) 각각 O(1) 평가 (커밋먼트 사용 시)
- **오른쪽**: H(r), Z(r) 각각 O(1) 평가

총 검증 시간: **O(1)**!

n개의 제약 조건을 상수 시간에 검증할 수 있습니다. 이것이 **간결함(Succinctness)**의 핵심입니다.

---

7. FFT: $O(n^2)$를 $O(n \log n)$으로

7.1 문제: 다항식 연산의 복잡도

라그랑주 보간으로 n개의 점에서 다항식을 구하면: $O(n^2)$
두 차수-n 다항식의 곱셈: $O(n^2)$

n = 2^20 (약 100만)이면 n^2 = 2^40 ≈ 1조 연산. 실용적이지 않습니다.

7.2 FFT (Fast Fourier Transform)

FFT (빠른 푸리에 변환)는 다항식의 **계수 표현** ↔ **평가 표현** 변환을 O(n log n)에 수행합니다[^6].

- **계수 표현**: $[a_0, a_1, a_2, \ldots, a_{n-1}]$
- **평가 표현**: $[P(\omega^0), P(\omega^1), P(\omega^2), \ldots, P(\omega^{n-1})]$

여기서 ω는 **n번째 원시근(primitive n-th root of unity)**입니다:

```
ωⁿ = 1, ωᵏ ≠ 1 (0 < k < n)
```

[^6]: Wikipedia. *Fast Fourier transform*. https://en.wikipedia.org/wiki/Fast_Fourier_transform

7.3 왜 원시근인가?

원시근 ω는 특별한 성질을 갖습니다:

```
Σₖ (ωⁱ)ᵏ = 0  (i ≠ 0 mod n)
```

이 성질 덕분에 FFT의 분할 정복이 가능합니다.

7.4 Cooley-Tukey 알고리즘

FFT의 핵심 아이디어 (1965년 발표, 실제로 가우스가 1805년 발견)[^7]:

```
P(x) = P_even(x²) + x · P_odd(x²)
```

여기서:

- P_even(x) = $a_0 + a_2x + a_4x^2 + \ldots$
- P_odd(x) = $a_1 + a_3x + a_5x^2 + \ldots$

분할: n점 FFT를 두 개의 n/2점 FFT로 분할
정복: 결과를 O(n)에 합침

```
T(n) = 2T(n/2) + O(n) = O(n log n)
```

[^7]: Cooley, J. W., & Tukey, J. W. (1965). *An algorithm for the machine calculation of complex Fourier series*. Mathematics of Computation, 19(90), 297-301.

7.5 NTT: 유한체에서의 FFT

일반 FFT는 **복소수**를 사용합니다. 복소수는 부동소수점 오차가 있습니다.

NTT (Number Theoretic Transform)는 유한체에서 FFT를 수행합니다[^8]:

- 복소수 대신 **유한체의 원시근** 사용
- 정수 연산만 사용 → **정확한 결과**
- ZKP에서는 항상 NTT 사용

조건: 유한체 F_p에서 NTT를 하려면:

```
p - 1 = 2^k × c  (k가 충분히 큼)
```

BN254, BLS12-381 등의 커브는 이 조건을 만족하도록 설계되었습니다.

[^8]: Pollard, J. M. (1971). *The fast Fourier transform in a finite field*. Mathematics of Computation, 25(114), 365-374.

7.6 ZKP에서 FFT의 역할

증명 생성 과정:

1. R1CS → QAP 변환: 라그랑주 보간 → **IFFT** (역 FFT)
2. A(x) × B(x) - C(x) 계산: 다항식 곱셈 → **FFT**

3. H(x) = (A×B-C) / Z 계산: 다항식 나눗셈

FFT 덕분에 증명 생성이 O(n log n)에 가능합니다.

---

8. 전체 파이프라인 요약

```
1. 고수준 코드
   def f(x):
       return x**3 + x + 5

2. 평탄화 (Flattening)
   sym_1 = x * x
   y = sym_1 * x
   sym_2 = y + x
   out = sym_2 + 5

3. R1CS
   A, B, C 행렬 + 솔루션 벡터 s
   (As) ⊙ (Bs) = Cs

4. QAP (라그랑주 보간)
   A(x), B(x), C(x) 다항식
   A(x) × B(x) - C(x) = H(x) × Z(x)

5. 다항식 커밋먼트 & 검증
   Schwartz-Zippel: 무작위 점에서 평가
   O(1) 검증 시간
```

8.1 복잡도 정리

단계	증명자	검증자
R1CS → QAP	O(n log n)	-
다항식 곱셈	O(n log n)	-
커밋먼트 생성	O(n)	-
증명 크기	-	O(1)
검증 시간	-	O(1)*

*공개 입력이 있으면 O(공개 입력 크기)

---

9. 실제 예제: 유한체에서의 계산

실제 ZKP에서는 모든 계산이 유한체에서 이루어집니다. Vitalik의 예제는 설명을 위해 실수를 사용했지만, 이는 부동소수점 오차를 야기합니다[^9].

유한체 F_641에서 같은 계산을 해봅시다:

```
p = 641  # 소수
s = [1, 3, 35, 9, 27, 30]  # 위트니스

# A 행렬의 첫 번째 열: [0, 0, 0, 5]
# 점: (1,0), (2,0), (3,0), (4,5)

# 라그랑주 보간 (mod 641)
# A0(x) 계산...

# 검증
assert (A0(1) * B0(1)) % p == (C0(1)) % p
# 모든 연산이 정확함!
```

유한체에서는:

- 나눗셈이 역원 곱셈으로 변환
- 모든 중간 결과가 0 ~ p-1 범위
- **부동소수점 오차 없음**

[^9]: RisenCrypto. *R1CS and QAP - From Zero to Hero with Finite Fields*. https://risencrypto.github.io/R1CSQAP/

---

10. 마치며: 왜 이 모든 것이 필요한가

산술화의 각 단계가 왜 필요한지 정리합니다:

단계	목적	결과
평탄화	복잡한 연산 → 단순한 게이트	곱셈 1개씩 분리
R1CS	게이트 → 대수적 제약	행렬 형태로 표준화
QAP	행렬 → 다항식	n개 제약 → 1개 방정식
라그랑주 보간	점 → 다항식	R1CS 제약을 인코딩
Schwartz-Zippel	다항식 동등성 → 점 평가	O(1) 검증
FFT/NTT	다항식 연산 최적화	$O(n^2) \rightarrow O(n \log n)$

다음 편에서는 **Zcash Sprout**을 다룹니다:

- 최초의 프라이버시 암호화폐 (2016)
- BCTV14 증명 시스템
- Trusted Setup 세레모니
- 보안 취약점과 교훈

시리즈 로드맵

1편: 비밀을 증명한다는 것 ✓
2편: 컴퓨터가 이해하는 수학 ✓
3편: 프로그램을 다항식으로 - 산술화 (현재) ✓
4편: Zcash Sprout - 최초의 프라이버시 암호화폐 (2016)
5편: Groth16의 마법 (2016)
...

References

1. Buterin, V. (2016). *Quadratic Arithmetic Programs: from Zero to Hero*. Medium. https://medium.com/@VitalikButerin/quadratic-arithmetic-programs-from-zero-to-hero-f6d558cea649
2. 0xPARC. *R1CS Explainer*. https://learn.0xparc.org/materials/circom/additional-learning-resources/r1cs%20explainer/
3. RisenCrypto. *R1CS and QAP - From Zero to Hero with Finite Fields*. https://risencrypto.github.io/R1CSQAP/
4. RisenCrypto. *zkSNARKs: R1CS and QAP*. https://risencrypto.github.io/zkSnarks/
5. Wikipedia. *Lagrange polynomial*. https://en.wikipedia.org/wiki/Lagrange_polynomial
6. Wikipedia. *Schwartz–Zippel lemma*. https://en.wikipedia.org/wiki/Schwartz%E2%80%93Zippel_lemma
7. Wikipedia. *Fast Fourier transform*. https://en.wikipedia.org/wiki/Fast_Fourier_transform
8. Gennaro, R., et al. (2013). *Quadratic Span Programs and Succinct NIZKs without PCPs*. EUROCRYPT 2013.
9. Cooley, J. W., & Tukey, J. W. (1965). *An algorithm for the machine calculation of complex Fourier series*. Mathematics of Computation, 19(90), 297-301.
10. Pollard, J. M. (1971). *The fast Fourier transform in a finite field*. Mathematics of Computation, 25(114), 365-374.
11. zkCamp. *You can't understand ZKPs without understanding polynomials*. https://www.zkcamp.xyz/blog/you-cant-understand-zkps-without-understanding-polynomials
12. RareSkills. *The Schwartz-Zippel Lemma and its application to Zero Knowledge Proofs*. https://rareskills.io/post/schwartz-zippel-lemma